ProofHire Technical Implementation Plan

Oct 31, 2026 · @Elikem

1. Overview and goals

ProofHire is a hiring screen that checks resume projects against the real code on GitHub, then has an AI interview panel pick the candidate apart from three perspectives. We have 2 hours and must show it working live in a 2 minute demo.

What the product does

1. HR pastes a job description. Each interviewer persona (project manager, staff engineer, associate engineer) enters what they expect from a candidate.
2. Candidates are loaded with resume text and GitHub links.
3. Forensics runs on every repo in an isolated sandbox and produces an evidence report.
4. Each candidate gets a relevance score, an authenticity score and a combined fit score. They are ranked.
5. The top candidates go to a panel of persona agents that review the resume and repo evidence, cross-examine each other, and produce one final report.
6. HR sees the ranked list and the final panel report per candidate. A separate panel room view shows the debate live, and exists mainly for the demo.

What the demo must show

Judges score real-time AI use, originality, execution, design and potential. Two sponsor track prizes also reward effective use of Terse Durable Actors (stateful apps) and Agent Relay (agent to agent). So the panel debate must be visibly live, with Terse as the stateful referee and Agent Relay as the channel the persona agents talk through.

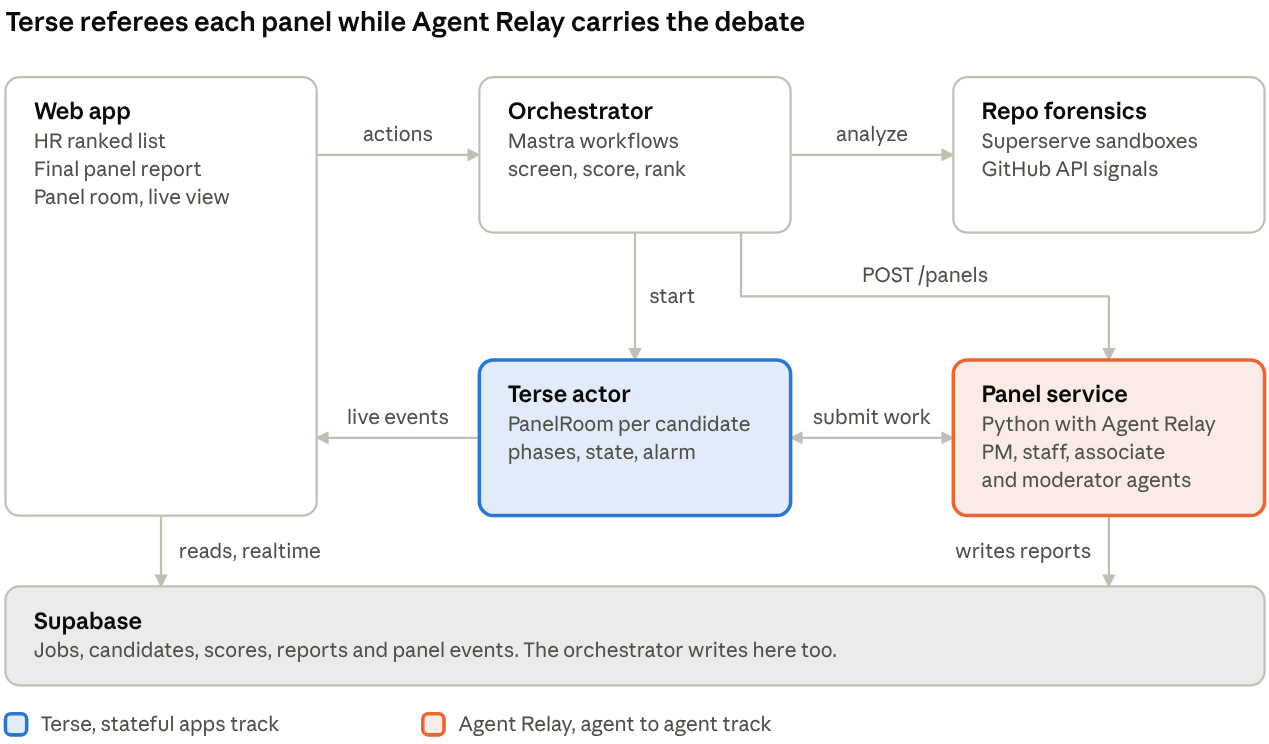
Scope for this build

- 5 seeded candidates for one job. The top 3 go to the panel.
- Resume input is plain text, not PDF parsing.
- No login, no multi-tenant security, no live interview with a human candidate.
- Screening and forensics results are precomputed before the demo. Only the panel debate for one candidate runs live.

Rules for the building agent

- Build in the order of section 4 (contracts) first. Every track codes against the same types and mocks the others until they are ready.
- Never call an LLM or any slow network request inside the Terse actor. The actor is a fast state referee only.
- Official docs win over this plan wherever they disagree. Section 13 lists what is unverified.
- Keep every track runnable alone with fixture data from the seed folder.
- Commit small and often. A working thin slice beats a polished half.

2. Architecture and repo layout



ProofHire architecture · 6 components, 2 sponsor tracks highlighted

The web app talks to the orchestrator, which runs forensics and starts each panel. In a panel, the Python agents debate through Agent Relay and report to the Terse actor, which referees the rounds and pushes live events to the room view. Everything durable lands in Supabase.

Sponsor roles

Tool	Job in ProofHire	Prize angle
Terse	One Durable Actor per candidate panel, holding the case file and refereeing the rounds one call at a time	Stateful apps
Agent Relay	A channel per candidate where the persona agents post findings and challenge each other	Agent to agent
Mastra	Screening workflows, scoring and the HTTP API that starts panels	Overall prizes
Superserve	Isolated sandboxes that clone and build untrusted candidate repos	Execution quality
Supabase	Single database, realtime updates for the HR views, scale pitch	Scale prize

Repo layout

```
proofhire/
  apps/web              Vite React frontend
  apps/orchestrator     Mastra workflows and HTTP API
  apps/actor            Terse PanelRoom actor
  packages/contracts    shared TypeScript types
  packages/forensics    repo analysis with Superserve
  services/panel        Python FastAPI service with Agent Relay
  seed/                 job, candidates, fixtures
  scripts/              smoke tests
  .env                  never committed
```

3. Environment and first spikes

Set up the tools and run five go or no-go spikes in the first 15 minutes. Each spike has a pass condition. If one fails, switch to its fallback in section 13 immediately and do not debug it.

Prerequisites

- Bun 1.4.2 or later and Node 20 or later
- Python 3.10 or later (the Agent Relay SDK requires it)

- Terse CLI installed with `bunx terse-cli@latest install`, plus an API key from app.useterse.ai under API keys
- Superserve CLI and API key from console.superserve.ai
- A Supabase project, a GitHub personal access token with public repo read, and an LLM provider key

Environment variables (one `.env` at the repo root, never committed)

```
LLM_API_KEY=
LLM_MODEL_FAST=
LLM_MODEL_SMART=
GITHUB_TOKEN=
SUPABASE_URL=
SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
SUPERSERVE_API_KEY=
TERSE_ACTOR_URL=
TERSE_API_KEY=
RELAY_WORKSPACE=
RELAY_API_KEY=
RELAY_BASE_URL=
PANEL_SERVICE_URL=http://localhost:8000
```

Spikes

#	Spike	Pass condition	Fallback
1	Deploy the counter template actor to Terse and call it with the generated client	Method call returns and the value persists across two calls	Run `terse actor dev` locally for the whole demo
2	Call the same actor from Python over HTTP	Python reads and writes actor state	Add a small TypeScript bridge endpoint that Python calls
3	Two Python agents exchange messages in an Agent Relay channel	Agent B receives Agent A's post in under 5 seconds	Keep the channel posts but also log every message to the actor

#	Spike	Pass condition	Fallback
4	Create a Superserve sandbox, clone a public repo, run `git log`	Command output returns to our code	Use the GitHub API only for git signals
5	Scaffold Mastra with `npm create mastra@latest`, run one agent call and one workflow	Agent returns structured JSON validated by zod	Call the LLM provider SDK directly with the same prompts

Run spike 1 first, because the actor contract in section 4 depends on what the SDK allows.

4. Data contracts

Put these in `packages/contracts/src/index.ts` and commit them before any track starts. The Python service mirrors them as pydantic models in `services/panel/contracts.py`. Any change is announced to everyone and made in this one place.

```
export type Persona = "project_manager" | "staff_engineer" |
"associate_engineer" | "moderator";
export type ReviewerPersona = Exclude<Persona, "moderator">;
export type Verdict = "advance" | "maybe" | "reject";

export interface JobSpec {
  id: string;
  title: string;
  description: string;
  // what each interviewer expects, typed in by that interviewer
  personaExpectations: Record<ReviewerPersona, string>;
}

export interface CandidateInput {
  id: string;
  name: string;
  githubUsername: string;
  resumeText: string;
}

export interface ProjectClaim {
  name: string;
```

```
  description: string;      // as written on the resume
  claimedTech: string[];
  claimedFeatures: string[]; // concrete claims we can check in code
  repoUrl: string | null;
  claimedDates: string | null;
}

export interface StepResult { attempted: boolean; ok: boolean; logTail:
string; }

export interface ClaimCheck {
  claim: string;
  verdict: "supported" | "partial" | "unsupported";
  evidence: string; // file path or short quote from the repo
}

export interface RepoReport {
  repoUrl: string;
  exists: boolean;
  isFork: boolean;
  ownerMatchesCandidate: boolean;
  commitCount: number;
  candidateCommitCount: number;
  firstCommitAt: string | null;
  lastCommitAt: string | null;
  activeDays: number;
  largestCommitShare: number; // 0 to 1, share of lines added by the biggest
commit
  linesOfCode: number;
  languages: Record<string, number>;
  hasTests: boolean;
  install: StepResult;
  build: StepResult;
  test: StepResult;
  claimChecks: ClaimCheck[];
  flags: string[];          // short machine readable red flags
  authenticityScore: number; // 0 to 100
}

export interface CandidateScore {
  candidateId: string;
  relevance: number;    // 0 to 100
  authenticity: number; // 0 to 100, average over projects
```

```
  fit: number;          // 0 to 100, see section 7
  rank: number;
  summary: string;
}

export interface CaseFile {
  candidateId: string;
  candidateName: string;
  job: JobSpec;
  resumeText: string;
  projects: ProjectClaim[];
  repoReports: RepoReport[];
  score: CandidateScore;
}

export interface Finding { text: string; evidence: string; } // evidence
cites a resume line or a repo flag

export interface PersonaReview {
  persona: ReviewerPersona;
  verdict: Verdict;
  score: number; // 0 to 100
  headline: string; // one sentence
  strengths: Finding[];
  drawbacks: Finding[];
  redFlags: Finding[];
}

export interface CrossExamNote {
  persona: ReviewerPersona;
  targetPersona: ReviewerPersona;
  type: "challenge" | "concede" | "add";
  text: string;
}

export interface PanelReport {
  candidateId: string;
  panelScore: number;
  recommendation: Verdict;
  summary: string;
  consensus: string[];
  disagreements: string[];
  questionsForHR: string[];
```

```
  reviews: PersonaReview[];
  crossExam: CrossExamNote[];
  completedAt: string;
}

export type Phase = "idle" | "independent" | "crossexam" | "synthesis" |
"done";

export interface PanelEvent {
  seq: number;
  at: string;
  kind: "phase" | "review" | "crossexam" | "message" | "timeout" | "report";
  persona?: Persona;
  payload: unknown;
}
```

Fixtures live in `seed/fixtures/` as JSON files named after these types, so every track can run alone.

5. Supabase schema

Run this in the Supabase SQL editor. Row level security stays off for the hackathon, and the service role key is only used server side. JSON columns hold the contract types from section 4 as they are.

```
create table jobs (
  id uuid primary key default gen_random_uuid(),
  title text not null,
  description text not null,
  persona_expectations jsonb not null,
  created_at timestamptz default now()
);

create table candidates (
  id uuid primary key default gen_random_uuid(),
  job_id uuid references jobs(id) on delete cascade,
  name text not null,
  github_username text not null,
  resume_text text not null,
  status text not null default 'new',
  created_at timestamptz default now()
);
```

```
create table repo_reports (
  id uuid primary key default gen_random_uuid(),
  candidate_id uuid references candidates(id) on delete cascade,
  repo_url text not null,
  report jsonb not null,
  created_at timestamptz default now(),
  unique (candidate_id, repo_url)
);

create table candidate_scores (
  candidate_id uuid primary key references candidates(id) on delete cascade,
  relevance int not null,
  authenticity int not null,
  fit int not null,
  rank int,
  summary text,
  projects jsonb,
  updated_at timestamptz default now()
);

create table panel_reports (
  candidate_id uuid primary key references candidates(id) on delete cascade,
  panel_score int not null,
  recommendation text not null,
  report jsonb not null,
  completed_at timestamptz default now()
);

create table panel_events (
  id bigserial primary key,
  candidate_id uuid references candidates(id) on delete cascade,
  seq int not null,
  kind text not null,
  persona text,
  payload jsonb,
  created_at timestamptz default now()
);
create index on panel_events (candidate_id, seq);

alter publication supabase_realtime add table candidates, candidate_scores,
panel_reports, panel_events;
```

`candidates.status` moves through `new`, `screening`, `screened`, `panel_running`, `panel_done`, so the HR table can show progress without extra queries.

6. Track A, repo forensics

Build `packages/forensics` as a TypeScript module that turns a repo URL and a list of resume claims into a `RepoReport`. Untrusted code only ever runs inside a Superserve sandbox, never on a developer machine.

Public function

```
export async function analyzeRepo(input: {
  repoUrl: string | null;
  candidateGithubUsername: string;
  claim: ProjectClaim;
}): Promise<RepoReport>;
```

Steps inside `analyzeRepo`

1. If `repoUrl` is null or the GitHub API returns 404, return a report with `exists` false, the flag `repo_missing` and authenticity 0.
2. Call the GitHub REST API with `GITHUB_TOKEN` for repo metadata (fork flag, owner, created date) and the full commit list (paginate, cap at 500). Compute commit count, commits by the candidate, first and last commit dates, active days, and the share of lines added by the largest commit.
3. Create a Superserve sandbox, then clone the repo with `git clone --depth 200`. From the landing page the SDK shape is `Sandbox.create({ name })`, `sbx.commands.run(cmd)` and `sbx.pause()`. Check the Superserve docs for the exact return types and how to destroy a sandbox when finished.
4. In the sandbox count lines of code per language with a simple `find` and `wc` script, detect test files, and read the README and the 15 largest source files.
5. Try `install`, `build` and `test` using the detected ecosystem (npm, pip, go, flutter). Each step gets a 120 second timeout and returns `ok` plus the last 40 log lines. A failed or skipped step is data, not an error.
6. Pull the README, the file tree and key file contents out of the sandbox, then run the claim checker outside the sandbox. It is one LLM call that receives the claimed features and the repo contents and returns a `ClaimCheck` per claim with a verdict and

evidence. It must say `unsupported` when it cannot find evidence and must never guess.

7. Compute flags and `authenticityScore` with the formula below, then return.

Authenticity formula

Start at 100 and apply these penalties, then clamp to 0 to 100.

Condition	Penalty	Flag
Repo is a fork with no candidate commits	50	`fork_no_work`
Repo owner is not the candidate and the candidate has no commits	30	`not_owner`
3 or fewer total commits	25	`few_commits`
Largest commit holds more than 80 percent of lines	20	`single_dump`
Active on 1 day or less	15	`one_day`
Build step failed	15	`build_fails`
Install step failed	10	`install_fails`
Each unsupported claim	10	`claim_unsupported`
Each partial claim	4	none

Concurrency and caching

- Analyze at most 5 repos at once.
- Write every finished report to Supabase `repo_reports` and also to `seed/fixtures/repo-reports/<candidate>.json`. The demo reads from these and never waits for a live sandbox.
- Provide a CLI, `bun run forensics <repoUrl> <githubUsername>`, that prints the report as JSON. Use it to test against the seed repos.

Done when the CLI produces a valid `RepoReport` for one genuine repo, one scaffold repo and one missing repo, and the scores clearly separate them.

7. Track B, Mastra screening and orchestration

Build `apps/orchestrator` with Mastra (TypeScript). It extracts projects from resumes, calls forensics, scores and ranks candidates, writes results to Supabase, and starts panels for the top candidates. Scaffold with `npm create mastra@latest orchestrator`, then follow the Mastra docs for agents, workflows and the server. The landing page shows `Agent` from `@mastra/core/agent`, `createWorkflow` from `@mastra/core/workflows` and `Mastra` from `@mastra/core`, with model strings such as `openai/gpt-5.5`. Use whichever provider key we hold through `LLM_MODEL_FAST` and `LLM_MODEL_SMART`.

Agents (each returns structured output validated with zod from the contracts)

- `projectExtractor` reads resume text and returns `ProjectClaim[]`. It must pull concrete checkable features into `claimedFeatures`, for example "consumes Kafka topics" or "JWT auth with refresh tokens".
- `relevanceScorer` receives the job description, the extracted projects and their repo reports. It returns a 0 to 100 relevance score and a two sentence rationale. Relevance counts only what the repo evidence supports, so an unsupported claim adds nothing.

Workflow `screenCandidate`

1. `extractProjects` calls `projectExtractor`.
2. `analyzeProjects` runs `analyzeRepo` for every project in parallel (concurrency 5), or loads the cached fixture when `USE_FIXTURES=true`.
3. `scoreRelevance` calls `relevanceScorer`.
4. `computeScore` sets `authenticity` to the mean of the project authenticity scores, weighted by lines of code, and sets `fit` to round(0.5 * relevance + 0.5 * authenticity). If any project is flagged `repo_missing` for a project the resume lists as a headline project, cap `fit` at 40.
5. `persist` writes `candidate_scores` and `repo_reports` to Supabase and sets `candidates.status` to `screened`.

Workflow `screenJob`

1. Load the job and its candidates.
2. Run `screenCandidate` for all candidates in parallel.
3. Rank by `fit` and write `rank`.
4. For the top N (default 3), call `startPanel(candidateId)`.

`startPanel(candidateId)`

1. Build the `CaseFile` from Supabase.
2. Call the Terse actor's `start(caseFile)` method with actor id equal to the candidate id.
3. POST to `PANEL_SERVICE_URL/panels` with `{ candidateId }`.
4. Set `candidates.status` to `panel_running`.

HTTP surface (a small Hono server alongside Mastra, port 4111 for Mastra Studio, port 4000 for this API)

Method and path	Purpose
`POST /jobs`	Create a job and its candidates from a JSON body or the seed files
`POST /jobs/:id/screen`	Run `screenJob`
`POST /candidates/:id/panel`	Start a panel for one candidate (used by the demo button)
`GET /health`	Liveness check

Optional extra Add one Mastra scorer that checks every finding in a panel report cites real evidence from the case file. The Mastra traces and scorer results make a good second demo screen.

Done when `POST /jobs/:id/screen` on the 5 seeded candidates fills `candidate_scores` and the genuine candidates rank above the inflated ones.

8. Track C, Terse panel actor

Build `apps/actor` as one Durable Actor class, `PanelRoom`, with one instance per candidate (actor id equals candidate id). It is the referee and memory of a panel. It stores the case file, accepts each persona's work, decides when the panel moves to the next phase, and broadcasts events to anyone connected. It never calls an LLM or any external service.

Start from the counter template

```
terse init panel --template counter
cd panel/actor && bun install
terse actor dev          # local dev server, logs show here
# in the client folder
terse actor generate     # regenerate after every actor change
```

From the quickstart, an actor extends `Actor` imported from `terse-sdk/actor`. Fields marked `@Persisted` survive between calls, fields marked `@Emittable` sync to connected clients, and `this.broadcast(...)` pushes a message to every connected socket. A socket message arrives in `onMessage(socket, message)`. Calls and socket events share one mailbox and run one at a time, which is exactly the guarantee this design relies on. The concepts page also describes an `@Interleave` decorator for long awaits. We should not need it because no method awaits anything slow.

State

Field	Type	Notes
`phase`	`Phase`	Starts `idle`
`caseFile`	`CaseFile or null`	Set by `start`
`reviews`	map of persona to `PersonaReview`	Round 1 results
`crossExam`	map of persona to `CrossExamNote[]`	Round 2 results
`report`	`PanelReport or null`	Set by `submitReport`
`events`	`PanelEvent[]`	Append only log, also gives late joiners a replay
`seq`	number	Event counter

If the SDK cannot persist nested objects directly, store each field as a JSON string and parse on read. Check the TypeScript SDK guide (docs.useterse.ai/reference/typescript-sdk.md) for the right SQLite or persisted field approach before writing the class.

Methods

Method	Behavior
`start(caseFile)`	Reset state, store the case file, set phase `independent`, emit a `phase` event, schedule the alarm for 90 seconds
`submitReview(review)`	Store by persona, replacing any earlier one, emit a `review` event. When all 3 personas have reviewed, set phase `crossexam`, emit a `phase` event, reset the alarm
`submitCrossExam(persona, notes)`	Store, emit a `crossexam` event. When all 3 personas have submitted, set phase `synthesis`, emit, reset the alarm
`postMessage(persona, text)`	Append a `message` event only. Used for the Agent Relay chatter shown in the panel room
`submitReport(report)`	Store, set phase `done`, emit a `report` event, clear the alarm
`getState()`	Return phase, case file, reviews, crossExam, report and the last 200 events
`onAlarm()`	Mark every persona that has not submitted in the current phase as timed out, emit a `timeout` event for each, then advance the phase as if they had submitted an empty result
`onMessage(socket, message)`	On a `subscribe` message, send the stored event log to that socket so a late viewer sees the full debate so far

Every write method is idempotent. The Python service may retry, so submitting the same persona twice must overwrite and never double count. Every method validates its input against the contract and returns a clear error for unknown personas or wrong phases, without throwing.

Events

Each event is a `PanelEvent` with an incrementing `seq` and an ISO timestamp. The actor broadcasts the event after storing it. The browser and the Python service both use `seq` to ignore duplicates.

Deploy and test

1. Run `terse deploy` from the actor project, create an API key in the Terse app, and put the actor URL and key into `.env`.
2. Write `scripts/actor-smoke.ts` that calls `start` with a fixture case file, submits 3 fake reviews, 3 fake cross exams and a fake report, and asserts the phase sequence.
3. Run the same flow from Python once to confirm spike 2.

Done when the smoke script passes against the deployed actor, a second client connected over websocket receives the events live, and killing the alarm path (submitting only 2 reviews) still advances the phase after 90 seconds.

9. Track D, Python panel service with Agent Relay

Build `services/panel` as a FastAPI app. It runs the persona agents, lets them talk to each other through Agent Relay channels, and reports their work to the Terse actor and to Supabase. This is the part judges see as agent to agent communication.

Setup

```
cd services/panel
python -m venv .venv && source .venv/bin/activate
pip install fastapi uvicorn httpx pydantic supabase agent-relay-sdk openai
uvicorn main:app --port 8000
```

The Agent Relay Python SDK has two modes. Orchestrate mode spawns Claude, Codex or Gemini agents into channels. Communicate mode gives your own agent a `Relay("Name")` client with `send`, `post` (to a channel), `inbox` and message event hooks, configured through `RELAY_WORKSPACE`, `RELAY_API_KEY` and `RELAY_BASE_URL`. Use Communicate mode. Each persona is our own LLM call wrapped with a `Relay` client, which is far faster to build and more reliable than spawning full coding agents. Read the SDK page (pypi.org/project/agent-relay-sdk) and agentrelay.com/docs first, because the exact method names above come from the package description.

Endpoints

Method and path	Behavior
`POST /panels`	Body `{ candidateId }`. Starts the panel as a background task and returns immediately

Method and path	Behavior
`GET /panels/{candidateId}`	Returns the current phase from the actor, for debugging
`GET /health`	Liveness check

Panel run

For candidate `c`, use a channel named `cand-<first 8 chars of c>` and these agents, each a class with its own `Relay` client named `pm-<short>`, `staff-<short>` and `assoc-<short>`, plus `moderator-<short>`.

1. Load the `CaseFile` from the actor with `getState()`. Fall back to Supabase if the actor is empty.
2. **Round 1, independent reviews.** Run the three reviewers concurrently with `asyncio.gather`. Each makes one LLM call with the persona prompt below and returns a `PersonaReview` as validated JSON. Then it posts its headline and top red flag to the channel with `relay.post(channel, text)`, calls `postMessage` on the actor with the same text, and calls `submitReview` on the actor.
3. **Round 2, cross examination.** Wait until the actor phase is `crossexam`, polling every second. Each reviewer reads the channel inbox to get the other two reviews, makes one LLM call that returns `CrossExamNote[]` (challenge, concede or add, with a short text), posts each note to the channel addressed to the target persona, and calls `submitCrossExam` on the actor.
4. **Round 3, synthesis.** Wait for phase `synthesis`. The moderator reads the channel history, the reviews and the notes, makes one LLM call with `LLM_MODEL_SMART`, and returns a `PanelReport`. It calls `submitReport` on the actor.
5. After `submitReport`, write the report to `panel_reports`, copy all events to `panel_events`, and set `candidates.status` to `panel_done`.

A reviewer that fails twice submits a placeholder review marked as timed out rather than blocking the room. Use `LLM_MODEL_FAST` for rounds 1 and 2 and aim for a full panel under 60 seconds.

Persona prompt template

```
You are the {role_title} on a hiring panel. Your lens is {lens}.
The interviewer who owns your seat wrote these expectations for the candidate
{expectations}

Job description
{job_description}

Case file (resume claims, forensic repo reports, scores)
{case_file_json}

Rules
- Judge only from the case file. Never invent facts about the candidate.
- Every strength, drawback and red flag must carry evidence, either a resume
line or a repo flag or finding.
- Be adversarial but fair. A claim the code does not support is a red flag,
not a drawback.
- Return JSON that matches the PersonaReview schema and nothing else.
```

Persona	`role_title`	`lens`
`project_manager`	Project Manager	ownership, scope delivered, impact, timeline consistency between resume and repo dates, communication quality of the README
`staff_engineer`	Staff Engineer	architecture depth, code quality signals, tests, whether complexity matches the claims, whether the build runs
`associate_engineer`	Associate Engineer	fundamentals, readability, learning ability, whether they could contribute to the team quickly

The moderator prompt asks for consensus points, real disagreements between personas, a final panel score and recommendation, and the 5 sharpest questions HR should ask if the candidate advances. Those questions must target gaps found in the evidence.

Done when `POST /panels` for one fixture candidate drives the actor through all phases and the final report lands in Supabase within 60 seconds.

10. Track E, frontend

Build `apps/web` with Vite, React, TypeScript and Tailwind. It uses `@supabase/supabase-js` for data and realtime and calls the orchestrator API for actions. Design quality is a scored criterion, so use a clean light theme, one accent color, generous spacing, and clear red, amber and green states for scores.

Screens

Route	Purpose	Key elements
`/`	Job setup	Job title, job description textarea, three expectation textareas labeled Project Manager, Staff Engineer and Associate Engineer, a candidate list loaded from the seed, a **Run screening** button
`/jobs/:id`	Ranked candidates	Table with rank, name, fit, relevance, authenticity, status chip and flag badges. Rows update live from `candidate_scores` and `candidates` through Supabase realtime. A **Run panel** button on the top 3
`/candidates/:id`	Final HR report	Panel recommendation and score at the top, the persona verdict cards, consensus and disagreements, questions for HR, then per project evidence with flags and claim checks. This is the only screen HR needs
`/candidates/:id/room`	Panel room (demo view)	Phase stepper (Independent, Cross-exam, Synthesis), three persona lanes plus the moderator, messages appearing as they happen, verdict chips, and a score reveal when the report lands

Panel room data hook

Write one hook, `usePanelEvents(candidateId)`, that returns an ordered, de-duplicated list of `PanelEvent` by `seq` and the current phase. It has two interchangeable backends.

1. **Supabase backend.** Subscribe to realtime inserts on `panel_events` filtered by candidate. Build this first because it is known to work. The Python service must insert each event as it happens for this to be live.
2. **Terse backend.** Connect to the Terse websocket for the candidate's actor, send `subscribe`, and append broadcast events. Check the Terse docs for the browser client and gateway URL. Switch the room to this backend as soon as it works, because it is the stateful apps showcase, and keep the Supabase one as an automatic fallback if the socket fails to connect within 3 seconds.

Details that make the demo land

- Persona lanes show an animated typing state from the moment a phase starts until that persona's event arrives.
- Cross-exam notes render as small reply bubbles pointing at the targeted persona.
- Red flags use a distinct red chip with the evidence text on hover.
- The score reveal at the end counts up from 0 to the panel score.
- Add a **Demo mode** toggle in the corner that disables real LLM calls and replays a recorded event log with realistic delays. It is the backup if the network or a service fails on stage.

Done when a fresh browser opened on the room route shows a full recorded panel replay, then shows a live panel for one candidate when the Run panel button is pressed.

11. Seed data and demo script

The demo only works if the data shows a clear contrast. A teammate owns this whole section and starts it in the first 15 minutes, in parallel with the spikes.

Seed files

- `seed/job.json` holds one job, for example Senior Full-Stack Engineer on a real-time data platform, with a realistic job description and one paragraph of expectations for each of the three personas.
- `seed/candidates/*.json` holds 5 `CandidateInput` files.
- `seed/fixtures/repo-reports/*.json` holds the forensic result for each candidate once Track A produces it.
- `seed/fixtures/panel-replay.json` holds a recorded event log of one full panel, captured from a real run, for Demo mode.

The five candidates

Candidate	Resume claim	Repo reality	Expected outcome
A, genuine strong	Real-time dashboard with websockets and a data pipeline	Real project with a long commit history, tests and a working build	Top rank, panel advances
B, genuine moderate	Solid web app with auth and a REST API	Real but smaller project, few tests	Rank 2 or 3, panel maybe
C, inflated (hero)	Distributed event streaming pipeline with Kafka and sub-second analytics	Scaffold repo with 2 commits, a long README and boilerplate only	Mid rank on resume, drops hard on authenticity, panel rejects
D, inflated	Machine learning recommendation engine	Fork of a public project with no candidate commits	Low rank, panel rejects
E, fully fake	Three impressive projects	Repo URLs return 404	Last rank, never reaches the panel

Use a teammate's real repos, with their permission, for A and B. Create the repos for C and D ourselves so we control exactly what is in them. Candidate C is the live demo subject, so write its resume to look excellent for the job and make the gap in its repo obvious but not cartoonish.

Two minute demo script

Time	What happens	What the judges see
0 to 20 s	State the problem, LLM-polished resumes full of projects nobody built. Paste the job description and the three persona expectations	Job setup screen
20 to 50 s	Show the ranked list, precomputed. Open candidate C and show the evidence, one commit dump and unsupported claims	Ranked table, then the evidence view

Time	What happens	What the judges see
50 to 100 s	Press **Run panel** on candidate C and switch to the room. Three personas review, cross-examine and the moderator concludes	Live panel room, phases advancing, messages appearing
100 to 120 s	Show the final HR report with recommendation, disagreements and questions to ask. Close on scale, every company that hires has this problem	Report screen

Name the sponsor tools in one sentence while the room is running. Terse holds each panel's state and referees the rounds, Agent Relay carries the agent to agent debate, Superserve sandboxes verified the repos, Mastra orchestrated the agents and Supabase stores everything.

Before going on stage

- [ ] Screening results for all 5 candidates are cached and load instantly
- [ ] One full live panel run on candidate C completed successfully in the last 30 minutes
- [ ] Demo mode replay works with the network off
- [ ] Terse actor, Python service and orchestrator are all running, and health endpoints answer
- [ ] A screen recording of a good run is saved locally

12. Milestones and acceptance checks

The 2 hours split into setup, parallel tracks, integration and polish. Each milestone has a check someone can run in under a minute. Do not start the next milestone's integration until the current check passes.

Minutes	Milestone	Acceptance check	Track
0 to 15	Monorepo, `.env`, Supabase schema, contracts committed, 5 spikes run	Spike table in section 3 all pass or fallback chosen. `packages/contracts` imports in every app	All

Minutes	Milestone	Acceptance check	Track
15 to 40	Fixtures and mocks in place	Every track runs standalone against `seed/fixtures`	All
15 to 60	Forensics module	CLI returns valid `RepoReport` for a real, a scaffold and a missing repo	A
15 to 60	Screening workflow	`screenCandidate` runs on a fixture and writes `candidate_scores`	B
15 to 60	Terse actor deployed	`scripts/actor-smoke.ts` passes against the deployed actor	C
15 to 60	Panel service	`POST /panels` completes a panel against the fixture case file and a local actor	D
15 to 60	Web app shells	All 4 routes render from Supabase fixtures, room route plays the replay	E
60 to 75	Real forensics feeds screening	Ranked list for 5 seeds with genuine candidates above inflated ones	A, B
75 to 90	Screening starts panels	`POST /jobs/:id/screen` ends with the top 3 in `panel_running` and reports appearing	B, C, D
75 to 90	Room goes live	Room view shows real events from a real panel through the Terse websocket, with Supabase fallback working	C, E
90 to 105	Polish and caching	All 5 candidates cached, replay file recorded from a real run, no console errors	All
105 to 115	Full rehearsal	Two complete 2 minute run-throughs, one of them with the network blocked in Demo mode	All
115 to 120	Buffer and submission	Submission form sent, repo public, recording saved	All

13. Fallbacks, risks and unverified assumptions

This plan was written from landing pages, the Terse quickstart and concepts pages, and the Agent Relay Python package description. The items below were not confirmed in detail. Verify each in the official docs before relying on it, and tell the team when reality differs.

Unverified assumptions

Assumption	Where it matters	How to verify
A deployed Terse actor can be called over plain HTTP from Python	Panel service in section 9	Spike 2. Read the Terse docs index at docs.useterse.ai/llms.txt
Terse browser websocket client and gateway URL work as described	Panel room in section 10	Read the TypeScript SDK guide and the chatroom example in the durable-actors repo
The Terse alarm scheduling method name and SQLite API	Actor in section 8	TypeScript SDK guide at docs.useterse.ai/reference/typescript-sdk.md
Agent Relay channels work with our own wrapped LLM agents using `Relay`, `post` and `inbox`	Panel service	Spike 3 and the SDK page on PyPI
Agent Relay may also have a TypeScript SDK	Could remove the Python service	Check agentrelay.com/docs and the AgentWorkforce/relay repo
Superserve method names and return types beyond `Sandbox.create`, `commands.run` and `pause`	Forensics in section 6	Spike 4 and docs.superserve.ai
Mastra API details for structured output and workflow branching	Orchestrator in section 7	Mastra docs and Studio at localhost 4111

Fallbacks

Risk	Fallback
Sandbox setup is slow or fails	Compute git signals from the GitHub API only and run the sandbox build check on 1 or 2 repos
Python cannot call the Terse actor	Add a small TypeScript bridge endpoint in the orchestrator that forwards calls to the actor
Agent Relay channel is flaky	Keep posting to the channel, but also send every message to the actor with `postMessage` so the room view never goes blank
Terse websocket fails in the browser	Room uses Supabase realtime on `panel_events`, switched automatically after 3 seconds
LLM is slow or rate limited	Use the fast model for rounds 1 and 2, cache everything before the demo, and fall back to Demo mode replay
GitHub rate limits	Always send `GITHUB_TOKEN` and read from cached fixtures during the demo
Deploy trouble before the demo	Run every service locally. Terse supports `terse actor dev` locally

Fairness note for the pitch

Scores come from evidence, not from school or employer names, and a human makes the final call. Candidates whose work is private should be offered a way to grant read access or walk the panel through it, otherwise they would be unfairly penalized.